

Chapter 2: Working With Branches

Modifying and Comparing Branches

- **Comparing Branches:** You used the `git diff` command to compare two branches, like so for the `documentation` and `front-end` branches. This helps you see the differences between the branches, such as new content added or changes made.
`git diff documentation front-end`
- **Renaming Branches:** To rename a branch, you used the `git branch` command with the `-m` flag, first providing the current branch name (`txt`) then the new name for the branch (`lw1959-text-bug`). This is useful for maintaining clarity and consistency in your branch names.
`git branch -m txt lw1959-text-bug`
- **Deleting Branches:** You learned how to delete branches using the `git branch` command with the `-d` or `-D` flag, like so for the `llm-upgrade` branch. The `-d` flag deletes a branch that has been merged, while the `-D` flag forcibly deletes a branch that hasn't been merged.
`git branch -D llm-upgrade`

Remotes

- **Remote Repositories:** These are repos stored in the cloud via services like GitHub, allowing access from any computer and facilitating collaboration.
- **Local vs. Remote Repos:** Local repos are stored on your computer, while remote repos are stored online or on a network.
- **Cloning Repositories:** You can clone a repo using the `git clone` command followed by the path or URL. For example:
`git clone https://github.com/datacamp/project`
- **Naming Remotes:** After cloning, Git names the remote as `origin` by default, but you can add more remotes with custom names using `git remote add`. For example:
`git remote add back-up https://github.com/datacamp/project`
- **Listing Remotes:** Use `git remote` to list remotes and `git remote -v` for detailed information.

Pulling From Remote

- **Fetching from a Remote:** You use `git fetch` to retrieve branches from a remote repository without merging them into your local repository. For example, to fetch from the `origin` remote:
`git fetch origin`

- **Merging Remote Changes:** After fetching, you need to merge the remote changes into your local branch using `git merge`. This integrates the fetched changes into your current branch.
- **Pulling from a Remote:** The `git pull` command combines fetching and merging in one step. It fetches the latest changes from the remote and merges them into your current branch. For example, to pull from the `origin` remote:
`git pull origin`
- **Handling Uncommitted Changes:** Git requires you to commit your local changes before pulling from a remote to prevent conflicts. If you have uncommitted changes, Git will abort the pull and prompt you to commit first.

Pushing Remotes

- **Pull vs. Push:** While `git pull` fetches and merges content from a remote repository into your local repo, `git push` does the opposite by merging your local changes into the remote repository.
- **Commit Before Push:** Ensure all local changes are committed before pushing to the remote.
- **Push Command:** Use `git push` followed by the remote name and the local branch name. For example:
`git push origin main`
- **Workflow:** The typical workflow involves pulling from the remote, working locally, committing changes, and then pushing updates back to the remote.
- **Handling Rejections:** If your push is rejected, it's often because your local branch is behind the remote branch. Pull from the remote to synchronize before pushing again.
- **Creating New Branches Remotely:** Push a local branch that doesn't exist in the remote using:
`git push origin new-branch`